

# Dapper

## Micro ORM sobre ADO.NET

---

Programación III — UTN Tucumán  
C# 14 / .NET 10

## El problema

---

Con ADO.NET puro, leer filas requiere convertirlas manualmente:

```
var contactos = new List<Contacto>();  
  
using var reader = command.ExecuteReader();  
while (reader.Read())  
{  
    contactos.Add(new Contacto  
    {  
        Id      = reader.GetInt32(0),  
        Nombre  = reader.GetString(1),  
        Apellido = reader.GetString(2),  
        Telefono = reader.GetString(3),  
        Email   = reader.GetString(4),  
    });  
}
```

- **Verboso** — igual para cada tabla
- **Frágil** — si cambia el orden de columnas, se rompe
- **Repetitivo** — el mismo patrón una y otra vez

## La solución: Dapper

---

**Dapper** extiende `DbConnection` para mapear filas a objetos automáticamente.

```
IEnumerable<Contacto> contactos = db.Query<Contacto>(
    "SELECT * FROM Contactos"
);
```

- El SQL lo escribimos nosotros
- El mapping de columnas → propiedades lo hace Dapper

## Dapper en el espectro de herramientas

---

| Herramienta   | Nivel        | Qué hace                                  |
|---------------|--------------|-------------------------------------------|
| ADO.NET       | Bajo         | Conexión, comandos, parámetros, readers   |
| <b>Dapper</b> | <b>Medio</b> | <b>SQL nuestro + mapping automático</b>   |
| EF Core       | Alto         | ORM completo: LINQ, tracking, migraciones |

*Dapper no genera SQL. No trackea objetos. No tiene migraciones.  
Solo hace una cosa, y la hace bien: **mapear filas a objetos**.*

## Instalación

---

```
dotnet add package Dapper
dotnet add package Dapper.Contrib      # opcional – CRUD sin SQL
dotnet add package Microsoft.Data.Sqlite
```

En un script `.cs` :

```
#:package Dapper@*
#:package Dapper.Contrib@*
#:package Microsoft.Data.Sqlite@*
```

## Cómo funciona el mapping

Dapper compara **nombres de columnas** con **nombres de propiedades**.

```
-- Tabla en la base de datos
CREATE TABLE Contactos (
  Id      INTEGER PRIMARY KEY,
  Nombre  TEXT,
  Apellido TEXT,
  Email   TEXT
);
```

```
// Clase en C#
class Contacto {
  public int    Id      { get; set; }
  public string Nombre  { get; set; } = "";
  public string Apellido { get; set; } = "";
  public string Email   { get; set; } = "";
}
```

Los nombres coinciden → Dapper conecta todo automáticamente.

## Métodos principales

---

| Método                                          | Cuándo usarlo                                         |
|-------------------------------------------------|-------------------------------------------------------|
| <code>Query&lt;T&gt;(sql)</code>                | Consulta que devuelve múltiples filas                 |
| <code>QuerySingle&lt;T&gt;(sql)</code>          | Exactamente un resultado (excepción si no)            |
| <code>QuerySingleOrDefault&lt;T&gt;(sql)</code> | Cero o un resultado ( <code>null</code> si no existe) |
| <code>Execute(sql)</code>                       | INSERT / UPDATE / DELETE                              |

## Query<T> y Execute

```
// Consultar
IEnumerable<Contacto> todos = db.Query<Contacto>(
    "SELECT * FROM Contactos"
);

// Contar
int total = db.QuerySingle<int>("SELECT COUNT(*) FROM Contactos");

// Buscar uno (puede ser null)
Contacto? c = db.QuerySingleOrDefault<Contacto>(
    "SELECT * FROM Contactos WHERE Id = @id",
    new { id = 5 }
);

// Insertar / actualizar / eliminar
db.Execute(
    "DELETE FROM Contactos WHERE Id = @id",
    new { id = 3 }
);
```



## Parámetros con @nombre

---

Siempre usar parámetros. Nunca concatenar strings con datos del usuario.

```
// ✅ Correcto – parámetro con @
db.Query<Contacto>(
    "SELECT * FROM Contactos WHERE Apellido = @apellido",
    new { apellido = "Díaz" }
);

// ❌ Incorrecto – SQL injection
db.Query<Contacto>(
    $"SELECT * FROM Contactos WHERE Apellido = '{apellido}'"
);
```

Se pueden pasar como **objeto anónimo**, **diccionario** o **instancia de clase**.

## Dapper.Contrib — CRUD sin SQL

---

Extiende la conexión con métodos CRUD listos para usar:

| Método                               | Qué hace                         |
|--------------------------------------|----------------------------------|
| <code>db.Insert(obj)</code>          | INSERT → devuelve el Id generado |
| <code>db.Insert(lista)</code>        | INSERT de múltiples objetos      |
| <code>db.Get&lt;T&gt;(id)</code>     | SELECT por clave primaria        |
| <code>db.GetAll&lt;T&gt;()</code>    | SELECT de todos                  |
| <code>db.Update(obj)</code>          | UPDATE                           |
| <code>db.Delete(obj)</code>          | DELETE                           |
| <code>db.DeleteAll&lt;T&gt;()</code> | DELETE de toda la tabla          |

## Dapper.Contrib — ejemplo

---

```
using Dapper.Contrib.Extensions;

// Insertar - devuelve el Id
long id = db.Insert(new Contacto {
    Nombre = "Elena", Apellido = "Estrada",
    Telefono = "555-0011", Email = "elena@example.com"
});

// Obtener por id
Contacto? elena = db.Get<Contacto>(id);

// Actualizar
elena!.Email = "elena.nueva@example.com";
db.Update(elena);

// Eliminar
db.Delete(elena);

// Todos
foreach (var c in db.GetAll<Contacto>())
    Console.WriteLine($"{c.Id}: {c.Nombre} {c.Apellido}");
```

## Convenciones de Dapper.Contrib

---

Deduce tabla y PK sin configuración:

| Qué deduce         | Cómo                                                               |
|--------------------|--------------------------------------------------------------------|
| Nombre de tabla    | Pluraliza la clase: <code>Contacto</code> → <code>Contactos</code> |
| Clave primaria     | Propiedad llamada <code>Id</code>                                  |
| Columnas ignoradas | Marcadas con <code>[Computed]</code>                               |

Si las convenciones no aplican → atributos:

```
[Table("mis_contactos")]
class Contacto {
    [Key]          // PK autogenerada (AUTOINCREMENT)
    public int Id { get; set; }

    [Computed]     // ignorada en INSERT/UPDATE
    public string NombreCompleto => $"{Nombre} {Apellido}";
}
```

## [Key] vs [ExplicitKey]

```
// PK autogenerada por la base
class Pedido {
    [Key]
    public int Id { get; set; }
    public string Descripcion { get; set; } = "";
}
```

```
// PK asignada por nosotros
class Producto {
    [ExplicitKey]
    public string Codigo { get; set; } = "";
    public string Nombre { get; set; } = "";
}
```

- [Key] → Dapper.Contrib omite el campo en el INSERT, la base lo genera
- [ExplicitKey] → Dapper.Contrib incluye el campo en el INSERT tal como viene

## Filtrado: servidor vs cliente

### ✓ En el servidor — correcto

```
var resultado = db.Query<Contacto>(  
    "SELECT * FROM Contactos " +  
    "WHERE Nombre > @nombre",  
    new { nombre = "Carlos" }  
);
```

Solo viajan las filas que coinciden.

```
var resultado = db.GetAll<Contacto>()  
    .Where(c => c.Nombre > "Carlos");
```

### ✗ En el cliente — ineficiente

Trae **todo** a memoria y filtra en C#.

Regla: filtrar con `WHERE` en SQL, no con `.Where()` en LINQ sobre `GetAll`.

## El flujo completo

```
Aplicación C#  
  ↓ escribe SQL + parámetros  
Dapper  
  ↓ envía la consulta  
ADO.NET / Provider (ej: Microsoft.Data.Sqlite)  
  ↓  
SQLite  
  ↑ devuelve filas  
ADO.NET / DataReader  
  ↑  
Dapper → mapea columnas → propiedades  
  ↑  
Objeto C# listo para usar
```

Dapper **no reemplaza** ADO.NET: lo usa internamente.

## Dapper.Contrib vs Dapper puro

---

| Situación                     | Qué usar       |
|-------------------------------|----------------|
| CRUD simple sobre una tabla   | Dapper.Contrib |
| Consultas con JOINS o filtros | Dapper puro    |
| INSERT con lógica especial    | Dapper puro    |
| Quiero ver el SQL exacto      | Dapper puro    |

Ambos pueden convivir en el mismo proyecto.



## ¿Cuándo usar Dapper?

---

| Situación                     | Herramienta                |
|-------------------------------|----------------------------|
| Control total sobre el SQL    | ADO.NET o Dapper           |
| Consultas complejas con JOINS | Dapper                     |
| CRUD simple + relaciones      | EF Core                    |
| Migraciones y tracking        | EF Core                    |
| Proyecto chico, SQL visible   | Dapper                     |
| Aprender qué pasa por debajo  | ADO.NET → Dapper → EF Core |

## Resumen

---

- Dapper extiende `DbConnection` con `Query<T>`, `Execute`, `QuerySingle<T>`
- El mapping es por **nombre de columna** ↔ **nombre de propiedad**
- Parámetros siempre con `@nombre` — nunca concatenar strings
- **Dapper.Contrib** agrega CRUD completo sin SQL manual
- Filtrar en la base ( `WHERE` ) es siempre mejor que `GetAll` + `.Where()`
- Dapper usa ADO.NET por debajo — no lo reemplaza